# CPE 231 – Algorithms (1/69)

## Homework II Solution

1. (10 points) Quickselect

   (a) (5 points) Given an input array $A[0 \ldots N-1]$, write pseudocode for a non-recursive version of Quickselect based on Lomuto partitioning.

**Algorithm** QuickSelect($A[0 \ldots N-1]$, $k$)

Input: Array $A[0 \ldots N-1]$ and integer $k$ $(1 \leq k \leq N)$
Output: $k$th-smallest element in $A[0 \ldots N-1]$

$l \leftarrow 0,\ r \leftarrow N-1$
$s \leftarrow$ LomutoPartition($A[l \ldots r]$)
**while** $s \neq k-1$ **do**
  **if** $s < k-1$ **then**
    $r \leftarrow s$
  **else**
    $l \leftarrow s+1$
  $s \leftarrow$ LomutoPartition($A[l \ldots r]$)
**return** $A[s]$

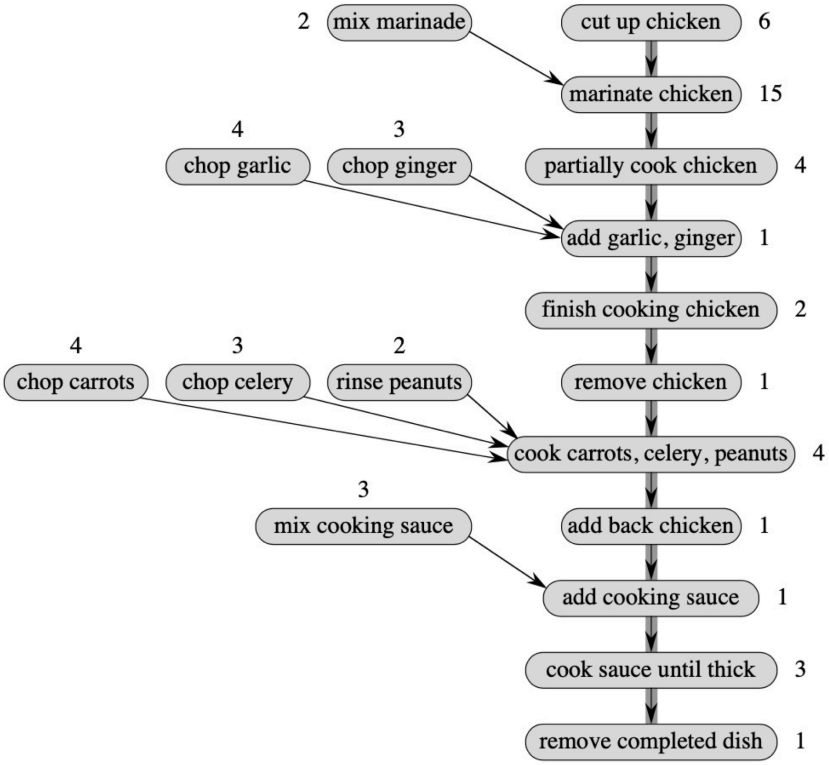
   (b) (5 points) Apply the algorithm in (a) to find the median of the list of numbers `9, 12, 5, 17, 20, 30, 8`. Trace your algorithm by showing how the search segment has changed in each iteration to verify its correctness.

***Soln.***

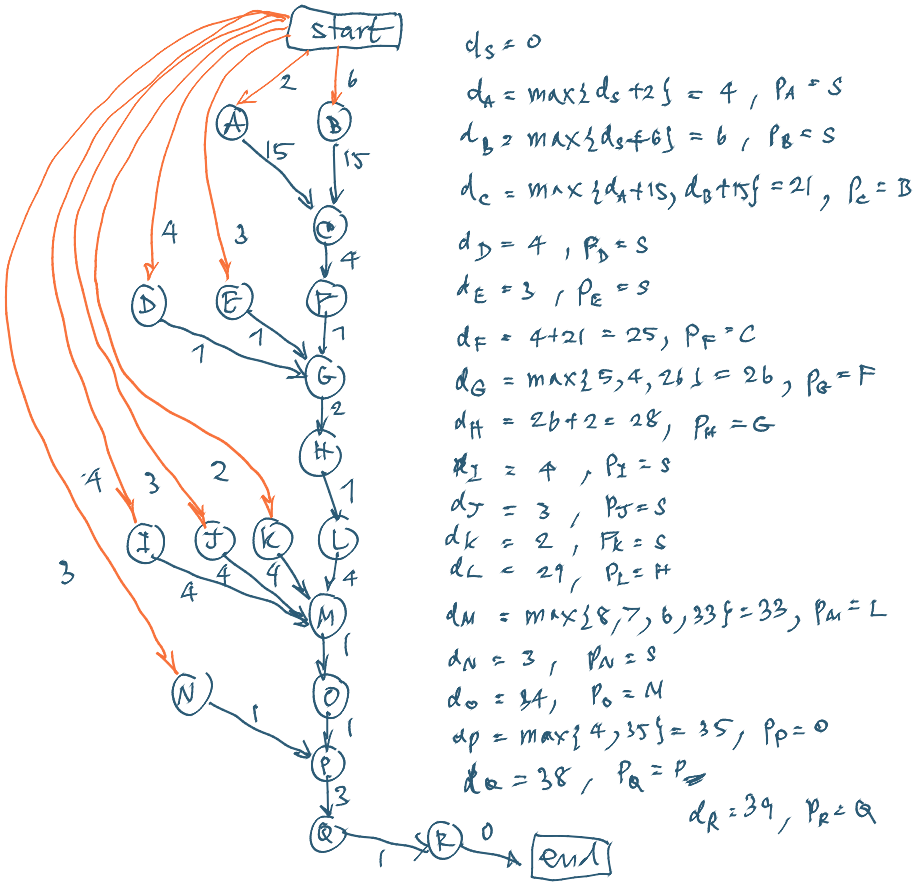
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| $s$ | $i$ | | | | | |
| **9** | 12 | 5 | 17 | 20 | 30 | 8 |
| $s$ | | $i$ | | | | |
| **9** | 12 | 5 | 17 | 20 | 30 | 8 |
| | $s$ | $i$ | | | | |
| **9** | 5 | 12 | 17 | 20 | 30 | 8 |
| | $s$ | | | | | $i$ |
| **9** | 5 | 12 | 17 | 20 | 30 | 8 |
| | | $s$ | | | | $i$ |
| **9** | 5 | 8 | 17 | 20 | 30 | 12 |
| 8 | 5 | **9** | 17 | 20 | 30 | 12 |
| | | | $s$ | $i$ | | |
| | | | **17** | 20 | 30 | 12 |
| | | | $s$ | | | $i$ |
| | | | **17** | 20 | 30 | 12 |
| | | | | $s$ | | $i$ |
| | | | **17** | 12 | 30 | 20 |
| | | | 12 | **17** | 30 | 20 |
| | | | **12** | | | |

□

2. (10 points) From the steps for making Kung Pao chicken below, determine the quickest possible time we can finish it by using the longest path DAG algorithm. Draw the input graph $G$ for the algorithm and show how the distance labels $d_v$ are updated in each iteration.




***Soln.*** First, we determine the linearized order of the input graph. Then, we convert the activity chart to DAG by adding start and finish nodes, and pushing back the task times to the incoming edges. The nodes has been labeled in the topological ordering so we can perform the edge relaxing step in the alphabetical order to determine the longest path from the start node to node $R$ as shown below.




From the algorithm output, the smallest time to finish the whole job is 39 minutes. The task sequence can be obtained by tracing back the previous node that leads to the longest path from the start node, which is B-C-F-G-H-L-M-O-P-Q-R. □

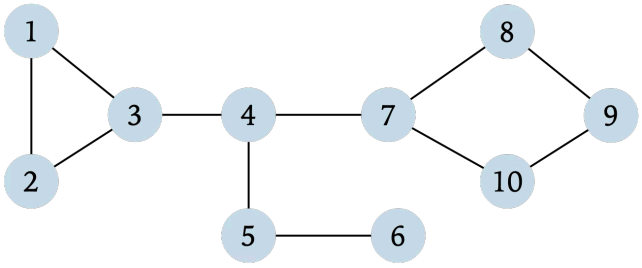
3. (20 points) Write a C program for DFS traversal on a unigraph.

   ▷ The program accepts an input file containing the number of nodes $N$ in the first line and a list of node pair $(i, j)$ as edges in the following lines.

   ▷ The node labels must be between 1 to $N$.

   ▷ Use the input to construct an adjacency list for the traversal. Then, print out the visit order and the finish order from the traversal.

   Submit the source code and the output of your program for the unigraph below:




4. (20 points) Modify your program in the previous question to implement Tarjan's algorithm to find articulation points in a graph. Submit the source code and the output of your program for the unigraph above.